Integração ACN-Sim e pandapower em malha fechada

Guia de trabalho para a tese

1 Resumo

- A documentação do ACN-Sim descreve a ligação ao pandapower como *sequencial*: a saída do ACN-Sim é a entrada do simulador de rede.
- O feedback da rede para o ACN-Sim está indicado como trabalho futuro.
- O pacote `acnportal` 0.3.3 instalado não contém código de rede. O código de integração está num repositório à parte, `caltech-netlab/acnportal-experiments`.
- Logo, a co-simulação em malha fechada tem de ser construída por nós. Isso é ao mesmo tempo um risco e uma contribuição da tese.
- A boa notícia é que a API do ACN-Sim tem os pontos de acoplamento certos. Validei um protótipo descartável (não é o esqueleto do projeto) em que o escalonador lê a tensão medida no pandapower no período anterior. Um dia de 288 passos de 5 minutos corre em cerca de 4 segundos.

Fontes. O paper ACN-Sim (arXiv 2012.02809), a secção de co-simulação com simuladores de rede, e o código-fonte do `acnportal` 0.3.3. A pesquisa na web indicou o repositório `acnportal-experiments` como local dos exemplos com pandapower e OpenDSS, também com acoplamento unidirecional. Não abri esse repositório nem a documentação em `acnportal.readthedocs.io`, por isso convém confirmá-los antes de assumir que não há nada reutilizável.

2 Pontos de acoplamento na API do ACN-Sim

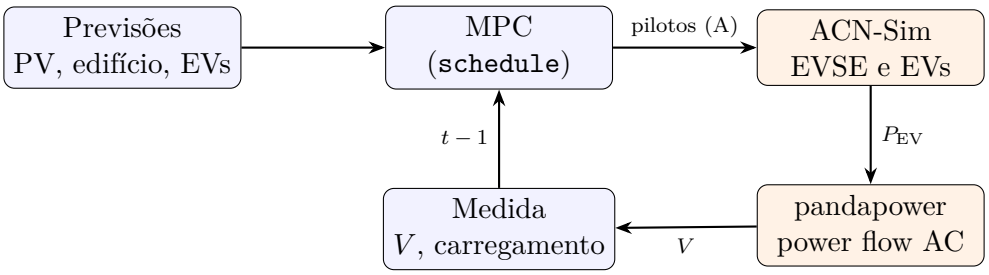
Elemento	Para que serve na co-simulação
`Simulator(network, scheduler, events, start, period)`	Ciclo principal. `period` é em minutos.
`BaseAlgorithm.schedule(active_sessions)`	Aqui vive o MPC. Devolve um dicionário de estação para lista de correntes (A).
`scheduler.max_recompute = 1`	Força a chamada do escalonador em todos os períodos. Sem isto, só é chamado quando há eventos.
`interface.current_time`	Índice do período atual.
`interface.last_actual_charging_rate`	Correntes (A) do período anterior, indexadas por *session id*.
`interface.evse_voltage(id)`, `evse_phase(id)`	Tensão e ângulo de fase de cada EVSE.
`interface.max_pilot_signal(id)`	Corrente máxima do EVSE.
`interface.remaining_amp_periods(s)`	Energia que falta, em A·períodos.
`interface.get_constraints()`	Matriz de restrições de corrente da rede de carregamento.
`sim.charging_rates`	Matriz estações × tempo com as correntes reais (A).
`Linear2StageBattery(cap, init, max_power)`	Bateria do EV, em kWh e kW.

Conversão de unidades, igual à usada internamente por `remaining_amp_periods`:

$$P_{\mathrm{kW}} = \frac{I_{\mathrm{A}}\, V_{\mathrm{EVSE}}}{1000}, \qquad \mathrm{A}\cdot\mathrm{períodos} = \frac{E_{\mathrm{kWh}}\cdot 1000}{V_{\mathrm{EVSE}}}\cdot\frac{60}{\mathrm{period}}.$$

O que o ACN-Sim não tem. O modelo `Battery` do ACN-Sim é o da bateria do EV. Não há bateria estacionária, PV nem carga de edifício. Tudo isso entra como séries externas e como modelos nossos, tanto no MPC como na "planta".

3 Arquitetura em malha fechada



Atraso de um período. O escalonador é chamado no início do período t e só vê o que se passou até $t-1$. Isto é realista (latência de medida) e liga-se à ideia de tratar atrasos como erro de modelo no paper de Pajic et al.

Duas formas de ligar o power flow.

1. **Dentro do escalonador.** Mais simples, mas `last_actual_charging_rate` só inclui as sessões ainda ligadas no período t. Uma sessão que terminou exatamente em t fica de fora e a carga é subestimada.

2. **Numa subclasse de `Simulator`** (recomendada). Sobrepõe-se `_store_actual_charging_rates`, que o ciclo chama logo após aplicar os pilotos. Aí usa-se a matriz `charging_rates` completa, sem perder sessões.

Testei a segunda: a tensão mínima do dia coincide exatamente (diferença máxima de 0) com a obtida a repetir o power flow a posteriori sobre `sim.charging_rates`.

```
class CoSimulator(acnsim.Simulator):
    def __init__(self, *args, grid, **kwargs):
        super().__init__(*args, **kwargs)
        self.grid, self.v_site = grid, 1.0

    def _store_actual_charging_rates(self):
        super()._store_actual_charging_rates()
        t = self._iteration
        kw = float(self.charging_rates[:, t].sum() * V_EVSE / 1000)
        self.v_site = self.grid.solve(t, kw)   # mesmo periodo, correntes exatas

class MPCWithGrid(BaseAlgorithm):
    def __init__(self):
        super().__init__()
        self.max_recompute = 1                 # chamar em todos os periodos

    def schedule(self, sessions):
        v_prev = self.sim.v_site               # medida do periodo t-1
        return self.mpc.solve(sessions, v_prev)  # dict {station_id: [A, ...]}
```

4 Pitfalls que encontrei

- A rede de carregamento precisa de pelo menos uma restrição. Sem ela, o `Simulator` falha ao validar (`constraint_matrix` é `None`).

- O ACN-Sim e o pandapower usam convenções diferentes: correntes por estação e fase de um lado, potência por barramento do outro. A conversão deve ficar num só sítio.
- A simulação termina quando a fila de eventos esvazia. No teste, o dia parou ao fim de 249 dos 288 passos. Para alinhar com séries de PV e edifício de 24 h, adicionar um evento final.
- O pacote exige `setuptools<81` (usa `pkg_resources`). Já está fixado no ambiente.
- Um schedule inviável levanta `InvalidScheduleError`. O MPC tem de respeitar as restrições que a `Interface` expõe e os níveis de piloto permitidos (por exemplo, o mínimo de 6 A do J1772).
- Cada passo custa cerca de 15 ms com o pandapower e numba, incluindo o escalonador trivial. Um MPC por cenários vai dominar o tempo.

5 Uma rede verosimil

5.1 O que não serve

A rede CIGRE LV do pandapower (`create_cigre_network_lv`) é a primeira ideia, mas não serve tal como está. Com as cargas base, a tensão mínima do alimentador comercial já é 0,912 pu, *sem* nenhum EV. No protótipo, um parque de 30 carregadores no barramento C9 baixou a tensão mínima para 0,75 pu. Esse valor não é realista e tornaria qualquer conclusão sobre o MPC irrelevante.

5.2 Proposta: rede paramétrica própria

Construir a rede com tipos standard do pandapower: fronteira de média tensão (20 kV, $S_{sc} = 250$ MVA), transformador `0.63 MVA 20/0.4 kV`, cabo `NAYY 4x150 SE` com comprimento e número de cabos em paralelo como parâmetros, cargas vizinhas num barramento, e o *site* (edifício, PV, bateria e carregadores) no fim do cabo. Duas configurações dão uma dimensão experimental, a força da rede.

Configuração A (forte): 80 m, 3 cabos em paralelo, edifício 40 kW

Carga EV (kW)	0	120	220	300
V no site (pu)	0,997	0,990	0,984	0,979
Transformador (%)	7	26	42	55
Cabo (%)	7	29	47	62

Configuração B (fraca): 300 m, 1 cabo, vizinhos 80 kW no barramento BT

Carga EV (kW)	0	40	80	120
V no site (pu)	0,976	0,959	0,940	0,921
Transformador (%)	20	27	34	41
Cabo (%)	23	45	69	93

Leitura dos resultados. Na configuração A a tensão quase não restringe: o que limita é o pico, o transformador e o cabo. É o caso típico de um parque de trabalho com ligação dedicada. Na B, a tensão desce abaixo de 0,95 pu por volta dos 60 kW de EVs, e o cabo chega a 93 % a 120 kW. É a configuração onde um controlador sensível à tensão tem algo a mostrar. O limite de 0,95 pu é uma escolha de projeto minha. O limite estatutário habitual (EN 50160) é ±10 % e deve ser confirmado.

Dimensionamento do site (a calibrar). 30 carregadores de 7,4 kW (222 kW), edifício entre 40 e 100 kW, PV de cerca de 80 kWp, bateria de cerca de 100 kW por 200 kWh. São números meus, por ajustar à literatura (por exemplo, o sistema de Torres et al. chega a 240 EVs e carregadores) e aos dados do ACN-Data.

5.3 Trifásico

O ACN-Sim modela correntes por fase, e o pandapower consegue um power flow desequilibrado com `runpp_3ph` e `create_asymmetric_load`. Os tipos standard não trazem dados de sequência zero. Sem eles falha logo com `KeyError` (por exemplo `x0x_max` e `r0_ohm_per_km`). Foi preciso fornecer:

- no `ext_grid`: `x0x_max` e `r0x0_max`;
- nas linhas: `r0_ohm_per_km`, `x0_ohm_per_km` e `c0_nf_per_km`;
- no transformador: `vk0_percent`, `vkr0_percent`, `mag0_percent`, `mag0_rx`, `si0_hv_partial` e `vector_group`.

Usei $r_0 = 4r_1$ e $x_0 = 3{,}5x_1$ como valores típicos assumidos. Estes valores são um pressuposto meu e têm de ser validados.

Resultado no site (cabo de 300 m, sem cargas vizinhas) com cargas diferentes em cada fase A, B e C:

Carga A/B/C (kW)	Total	V_A	V_B	V_C	Desequilíbrio (%)
20/20/20	60	0,975	0,975	0,975	0,00
30/20/20	70	0,948	0,984	0,979	0,57
45/0/0	45	0,876	1,043	1,020	2,51

Concentrar a carga numa fase derruba a tensão dessa fase (0,876 pu com apenas 45 kW) e faz subir as outras duas, por deslocamento do neutro. Isto reforça o argumento do paper ACN-Sim sobre o equilíbrio de fases, e é um bom motivo para o MPC saber em que fase está cada carregador. Um caso mais extremo (100 kW numa só fase, com cargas vizinhas desequilibradas) não convergiu. A não-convergência deve ser tratada como evento na simulação, e não como erro.

6 O que o protótipo mostrou

O protótipo descartável (guardado fora do projeto) correu um dia com 30 carregadores, na rede CIGRE LV, que sabemos ser não realista. Serve só para provar a ligação.

	Sem feedback	Escalonador lê a tensão
Tensão mínima (pu)	0,750	0,794
Pico de carga (kW)	175	161
Energia entregue / pedida (kWh)	563 / 563	403 / 563

O feedback tem efeito mensurável no comportamento da rede e custa energia não entregue, que é o compromisso que o MPC vai gerir de forma otimizada. Estes não são resultados científicos: o escalonador é uma heurística, e a rede não é realista.

7 Próximos passos

1. Abrir `acnportal-experiments` e a documentação online para ver o que reutilizar (notebooks com pandapower e OpenDSS).

2. Fixar a rede paramétrica (configurações A e B) e validar a calibração com o professor.
3. Reproduzir o case study sequencial do paper com pandapower, antes de ligar o feedback.
4. Só depois, o esqueleto do projeto.